

Instructions to deploy our Python Dash apps to cPanel

Jessica M. O'Loughlin (github.com/jessieolough)

With thanks to Alain Forrester (IS Applications, The University of Edinburgh) and Colin McLaren (SBS Computing, The University of Edinburgh) for their support to get our Dash apps deployed

Contents

[Background information](#)

[Dash app layout and requirements](#)

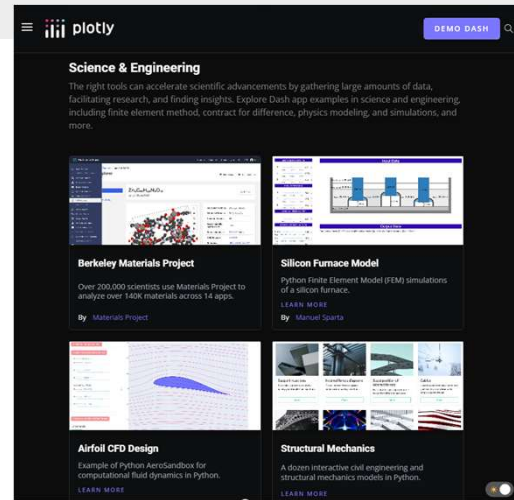
[Deploy initial instance of Dash App/Reports to cPanel](#)

[Upload additional Reports to cPanel](#)

[Troubleshooting Guidance](#)

Background information

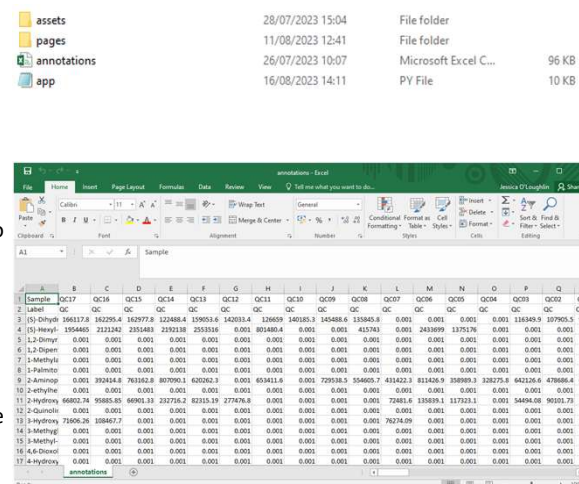
- **Python Dash Apps**
 - These are Python-based applications that allow you to interact with your data
 - <https://towardsdatascience.com/dash-for-beginners-create-interactive-python-dashboards-338bfc66ffa4>
- **EdinOmics: Metabolomics report app**
 - In summary, this app reads the same .csv as you would put into MetaboAnalyst
 - The data is processed and passed through various data analysis techniques
 - In our case, the user can choose which samples are included in the analyses, interact with the plots, and download their results.



Explore examples of Dash apps here:
<https://plotly.com/examples/>

Dash app layout and requirements: Prepare your files for deployment

- **Basics required to run a multi-page Dash app**
 - **annotations.csv**
 - File containing the metabolomics peak intensity data to be analysed and presented
 - **app.py**
 - Main application file
 - Reads the data + initially processes it according to the samples the user specifies
 - **assets**
 - Contains files to style the app, including .png (images), .ico (web icon), and .css (text styling) files
 - **pages**
 - Contains .py (Python) files which read the data processed by app.py and perform their respective data analyses
 - These are only active when the page is open by the user in the app



Dash app layout and requirements: Overall Script Structures

app.py (simplified)

[illegible]

Define App

Make sure that `/ExampleDevelopment` is set to an appropriate name for your project AND corresponds to the path set in `passenger_wsgi.py`

Set Up Log in Access details

Read corresponding AccessDetails.txt file with username and password information + set as authentication method



PEXAMPLE_AccessDetails - Notepad

File Edit Format View Help

```
{
  "example_username": "example_password"
}
```

Set Up Navigation Bar, where users can access each page of the report

"pexample/" in the `.startswith()` function should correspond to the module name of each page

Define the body_app

Important that the navbar (where users can select pages) and dash.page_container (where the selected pages are presented) are included

Define the app.layout and initialise the app

Dash app layout and requirements: Overall Script Structures

Pages (simplified)

[illegible]

Upload and create relevant data to initiate the report for the user to interact with and select data

Create the layouts for each tab (can have multiple per page)

Define the overall layout of the page

If tabs are used, these should be referenced here in `dbc.Tabs([ ])`

- Use callback functions to facilitate interactive elements of the report and background calculations

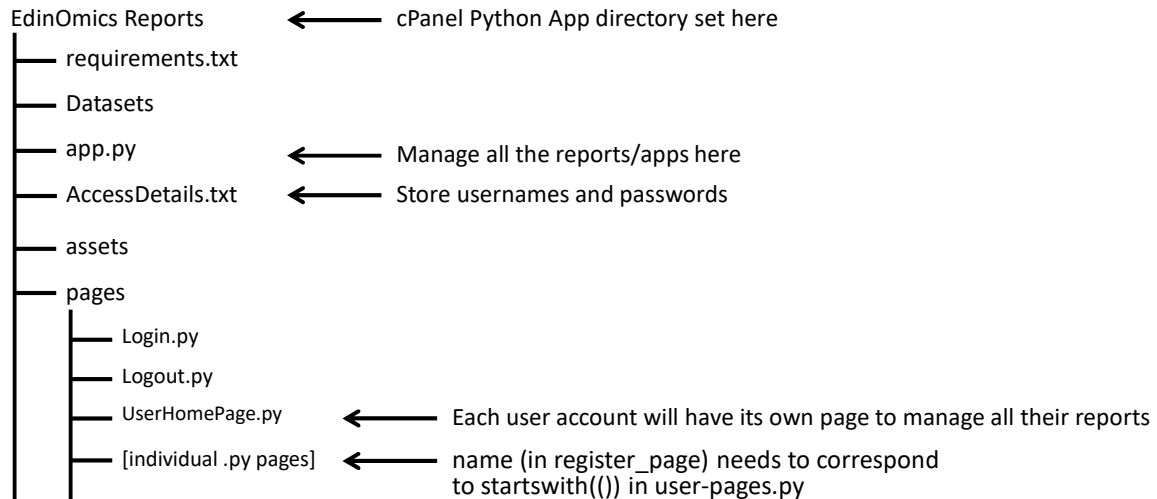
Register the page

Ensure that the module name prefix corresponds to that in the `.startswith()` function from `app.py`'s `navbar`

Important:

Make sure that the ids in the layout and callbacks are unique to each report (Do not directly copy and paste scripts for different reports without changing this)!

Dash app layout and requirements: Dash App Directory Structure



Dash app deployment requirements

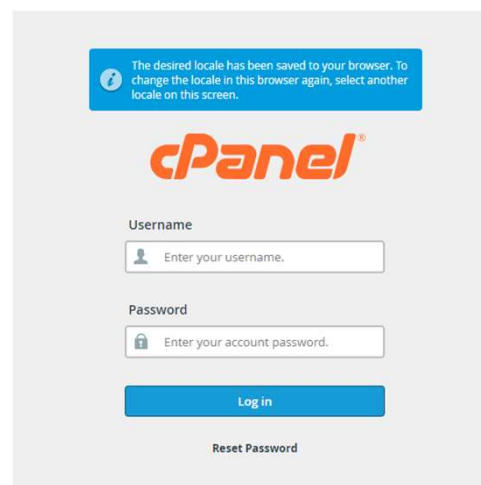
- To deploy our app on the cPanel server, additional files are required to facilitate this:
 - requirements.txt
 - Contains the Python modules required by the app to run and perform data processing/analysis
 - passenger_wsgi.py
 - Links our app to the cPanel server
 - Note: cPanel automatically generates a version of this when you initially upload your app. However, our own version (in the image to the right) is required to run the Dash app and ultimately needs to replace cPanel's own version
 - passenger.log
 - This is set whilst we deploy the app in cPanel
 - It is the main way error messages can be read from our app, which allows us to troubleshoot problems when they arise
 - There are also other files that cPanel generates (e.g. `__pycache__`, `public`, and `tmp` folders), but those can be left as they are.
 - Add `dash.Dash(__name__)` and `app.run_server(host='0.0.0.0', port=5000, debug=True)` to `app.py`

Deploy Dash app to cPanel: Upload your files

- Compress your app files as a zip file
 - This will make them easier to load into cPanel and keep the files in their correct structure
- Note: Steps in this process overwrite certain files, so ensure that you have backups of all your files before proceeding

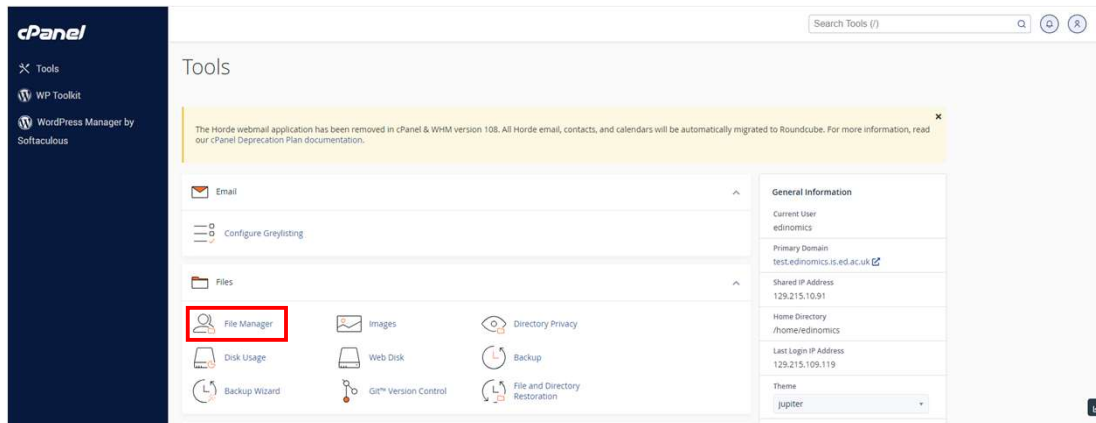
Deploy Dash app to cPanel: Log into cPanel

- Log into cPanel
 - <https://chostt.is.ed.ac.uk:2083/logout/?locale=en>



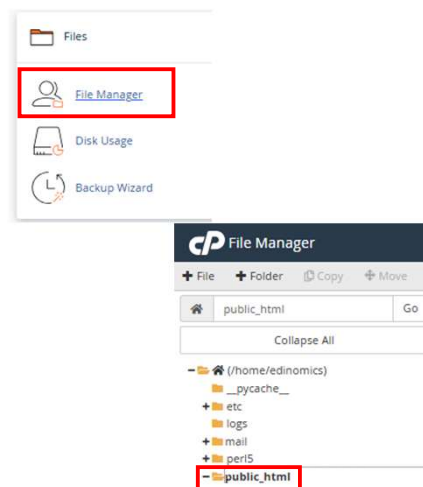
Deploy Dash app to cPanel: Navigate to cPanel File Manager

- Go to Tools > Files > File Manager



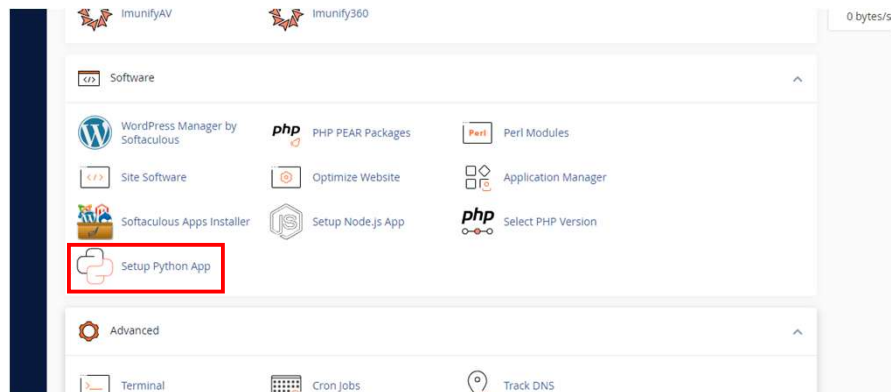
Deploy Dash app to cPanel: Upload and extract files in public_html

- Upload files into your desired folder within the public_html directory
 - Easiest to upload zip file + extract it in cPanel



Deploy Dash app to cPanel: Navigate to Setup Python App

- Go to Tools > Software > Setup Python App



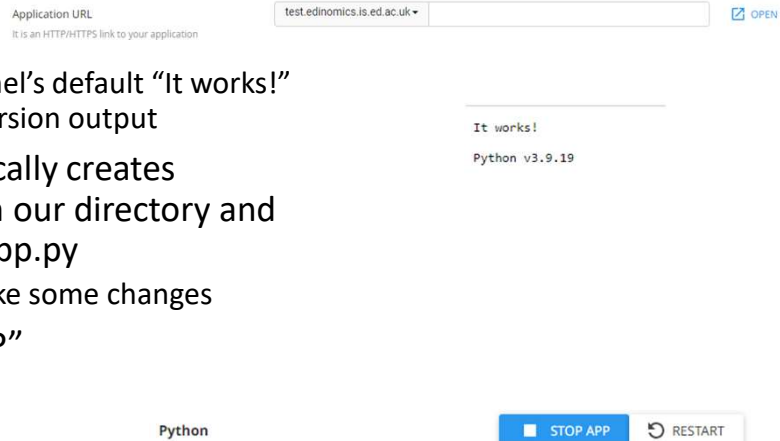
Deploy Dash app to cPanel: Create Python App

- Select “CREATE APPLICATION”
- Set up the app similar to the image
 - Python version = 3.9.19
 - Application root = [directory to app.py]
 - Application URL = [leave blank]
 - Startup file = app.py
 - Entry point = app
 - Passenger log file = /home/edinomics/logs/passenger.log
- Select “CREATE”



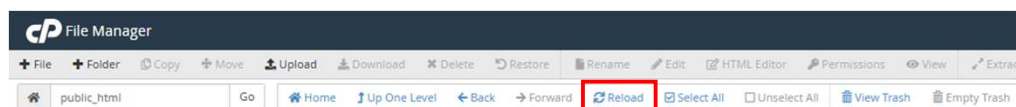
Deploy Dash app to cPanel: Initial checks

- Select “OPEN”
 - Should see cPanel’s default “It works!” + the Python version output
- cPanel automatically creates additional files in our directory and overwrites our app.py
 - We need to make some changes
- Select “STOP APP”



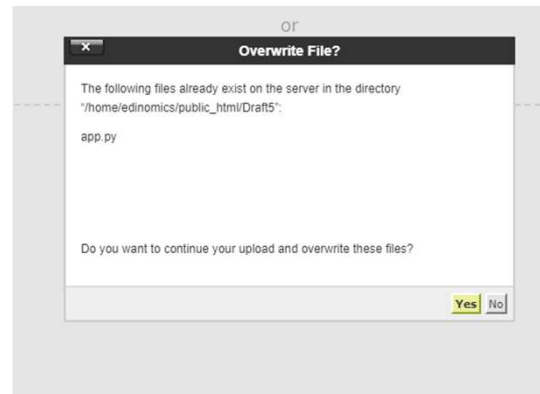
Deploy Dash app to cPanel: Return to cPanel File Manager

- Go back to your web tab containing the cPanel File Manager
- Refresh the page



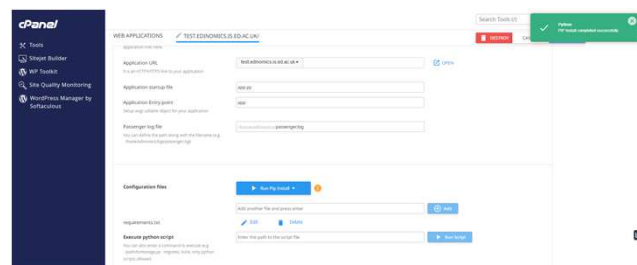
Deploy Dash app to cPanel: Re-upload app.py

- Re-upload your app.py file
 - Overwrite the one currently in your folder
- Right-click app.py and select “View” to make sure this has been done properly



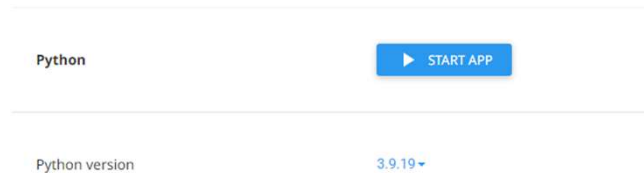
Deploy Dash app to cPanel: Install Python packages with requirements.txt

- Return to your Python app tab
- In the Configuration files section, type “requirements.txt” and press the enter key on your keyboard
 - This will upload the requirements.txt file in your folder
- Select “Run Pip Install”, which will drop down requirements.txt
- Select this requirements.txt and wait until the modules have been successfully installed



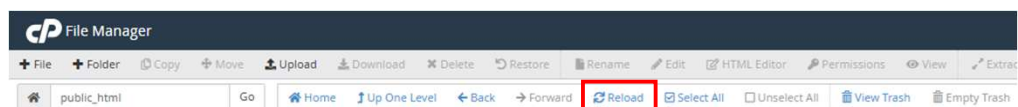
Deploy Dash app to cPanel: Re-start Python App

- Select “START APP”



Deploy Dash app to cPanel: Return to cPanel File Manager

- Go back to your web tab containing the cPanel File Manager
- Refresh the page



Deploy Dash app to cPanel: (re-)upload your passenger_wsgi.py file

- Upload your passenger_wsgi.py file
 - Overwrite the one currently in your folder, which was automatically generated/overwritten by cPanel

Deploy Dash app to cPanel: Restart Python App

- Return to your Python app tab
- Select “RESTART”
- Select “OPEN”
 - You should now be able to see your app successfully deployed
- Interact with the app to make sure that it works properly

Upload Additional Reports to cPanel: Files you will need to update

EdinOmics Reports

requirements.txt	
Datasets	← If adding to an existing account, will need to update
app.py	← If you are adding an additional user account, will need to update
AccessDetails.txt	← If you are adding an additional user account, will need to update
assets	
pages	
Login.py	
Logout.py	
UserHomePage.py	← If adding to an existing account, will need to update
[individual .py pages]	← Need to add additional results and/or methods pages (as required)

Upload Additional Reports to cPanel: Files you will need to update: Input Data

Metabolomics Data

Samples and groups must be labelled "Sample" and "Label" respectively

	A	B	C	D	E	F	G	H	I	J
1	Sample	A Ext 1	A Ext 2	A Ext 3	A Int 1	A Int 2	A Int 3	Blank01	Blank02	Blank03
2	Label	A Ext	A Ext	A Ext	A Int	A Int	A Int	Blank	Blank	Blank
3	(S)-Adenc	0.001	0.001	166899	0.001	178408.2	0.001	992647.3	0.001	0.001
4	(S)-Carbo	303374.1	215336.9	266310.2	0.001	0.001	0.001	0.001	0.001	0.001
5	(S)-Hexyl-	200957.2	178056	165733.8	0.001	55653.89	65993.74	0.001	0.001	0.001
6	(S)-Tauroc	1.34E+07	1.19E+07	1.20E+07	1.24E+07	1.19E+07	1.26E+07	0.001	0.001	0.001
7	(S)-Tauroc	0.001	0.001	0.001	0.001	0.001	0.001	0.001	0.001	0.001

Metabolomics Data's

corresponding Sample and Label assignments

Allows users to select the samples they want to analyse without all the metabolomics data needing to be uploaded at once (improves app performance)

Sample	Label
A Ext 1	A Ext
A Ext 2	A Ext
A Ext 3	A Ext
A Int 1	A Int
A Int 2	A Int
A Int 3	A Int
Blank01	Blank
Blank02	Blank
Blank03	Blank
B Ext 1	B Ext
B Ext 2	B Ext
B Ext 3	B Ext
B Int 1	B Int
B Int 2	B Int
B Int 3	B Int
Control Ex Control Ext	
Control Ex Control Ext	
Control Ex Control Ext	
Control In Control In	
Control In Control In	
Control In Control In	

Upload Additional Reports to cPanel: Files you will need to update: app.py

☐ = To Update if adding a new user account

Can use <https://pinetools.com/random-string-generator> to randomly generate 70-80 character unique href (do not include Symbols)

```
def login_button_click(n_clicks, username, password):
    if n_clicks > 0:
        if VALID_USERNAME_PASSWORD.get(username) is None:
            return "Invalid Username or Password"
        #If correct login details provided: direct the user to their own home page to access their project pages
        # Add in additional users as 'elif' arguments
        if VALID_USERNAME_PASSWORD.get(username) == password:
            login_user(User(username))
            if current_user.get_id() == "ExampleUsername":
                children = ["Login Successful ", dcc.Link("View Reports", href="/AddYourOwnAccountHef")]
                ☐ elif current_user.get_id() == "AlternativeExampleUsername": #Just put in as an example of how to add more
                children = ["Login Successful ", dcc.Link("View Reports", href="/AddYourOwnAlternativeAccountHef")]
            else:
                children = "Incorrect Username or Password. Is issue persists, contact EdinOmics for assistance."
            return children
        children = "Incorrect Username or Password. Is issue persists, contact EdinOmics for assistance"
    return children
```

Upload Additional Reports to cPanel: Files you will need to update: AccessDetails.txt

Can use <https://pinetools.com/random-string-generator> to randomly generate ~20 character password

☐ = To Update if adding a new user account

```
{
    "ExampleUsername": "ExamplePassword",
    ☐ "AlternativeExampleUsername": "AlternativeExamplePassword"
}
```

Note: commas should be used to separate different login Username:Password pairs EXCEPT for the last pair in the list (presence of a comma here crashes the app)

Upload Additional Reports to cPanel: Files you will need to update: Results Page

```
#TODO: Update for each individual user account
accountIndexPage = "/AddYourOwnAccountHref"
#TODO: Update for each individual page
page_name = "Experimental Results Example"

mainDataset = "ExampleData-MetabolomicsResults.csv"
#Some Metabolomics datasets can be HUGE and there may be cases
# --> to improve webpage speed, used sampleLabels to initial
# --> user can use these to select which samples will be analysed
sampleLabels = "ExampleData-SamplesAndLabels.csv"

dash.register_page(_name_,
                    name = page_name,
                    #TODO: Update for each individual page
                    path="/AddYourOwnResultsPath",
                    order = 1
)

#TODO: Update for each individual project
project_no = "ProjectNo"
```

```
dbc.Row([
    dbc.Col([
        dbc.Spinner(
            children=[
                dcc.Graph(id = "histogram_raw_ExampleReportSuffix"),
            ],
            size = "lg",
            color = "primary",
            fullscreen = True
        ),
    ]),
])
```

Can use <https://pinetools.com/random-string-generator> to randomly generate 70-80 character unique path (do not include Symbols)

The link matched to the accountsIndexPage object should correspond to the href for that user in app.py

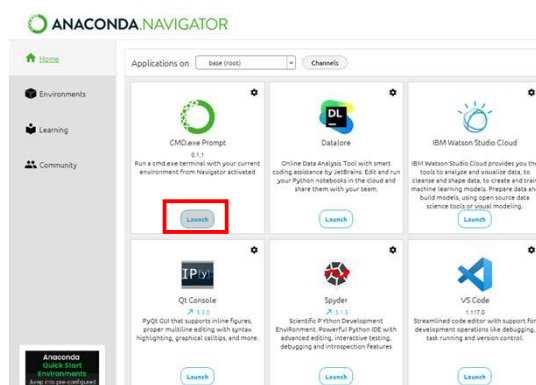
The name matched to page_name should correspond to the name within the if page["name"].startswith(()) list in UserHomePage.py

- = To Update if adding a new user account (can copy and paste existing methods-page.py file for the same user account and only make the changes in orange)
- = To Update if adding only additional reports to an existing user account

Upload Additional Reports to cPanel: Test New/Updated Report Locally first

This is recommended as this should catch most errors which could potentially crash the deployed Metabolomics Reports

- 1) Open Anaconda
- 2) Launch "CMD.exe Prompt"



Upload Additional Reports to cPanel: Open Local Conda Environment (simulates deployment)

Open the dash_app_login_env (simulates the environment the Dash Apps are deployed in)

- 1) Use "cd" to set the working directory to the folder containing app.py
 - If you are working in a different Computer drive to what Anaconda initially opens with, you will need to type the "Drive letter + :" (e.g. "C:") before you can set the new working directory
- 2) Optional: Type "conda info --envs" to get the exact names of your conda environments
- 3) Type "conda activate dash_app_login_env" to activate this environment
 - Should see the (base) change to (dash_app_login_env)

```

C:\WINDOWS\system32\cmd.exe
Microsoft Windows [Version 10.0.26100.8246]
(c) Microsoft Corporation. All rights reserved.

(base) C:\Users\1997024\OneDrive\Documents>cd C:\Python-Dash-App-and-cPanel-workflow-to-automate-metabolomics-data-analyses-and-visualisation\AppCode\ExampleHomeDirectoryForApp

(base) C:\Python-Dash-App-and-cPanel-workflow-to-automate-metabolomics-data-analyses-and-visualisation\AppCode\ExampleHomeDirectoryForApp>conda info --envs
# conda environments:
#
base                  *  C:\Anaconda
dash_app_login_env    C:\Users\1997024\OneDrive\Documents\condaenvs\dash_app_login_env

(base) C:\Python-Dash-App-and-cPanel-workflow-to-automate-metabolomics-data-analyses-and-visualisation\AppCode\ExampleHomeDirectoryForApp>conda activate dash_app_login_env

(dash_app_login_env) C:\Python-Dash-App-and-cPanel-workflow-to-automate-metabolomics-data-analyses-and-visualisation\AppCode\ExampleHomeDirectoryForApp>
  
```

Upload Additional Reports to cPanel: Open Local Conda Environment (simulates deployment)

Start the app and test it/fix bugs before officially deploying it

- 1) Type "python app.py" to run the app
- 2) The output should contain "Dash is running on http:..."
 - If not: you will need to go through whichever error messages pop up and fix them
- 3) Even if the app is running, this does not mean that it is free of errors!
 - You will need to click/paste the generated http... link into your browser to view your reports
 - Error messages may pop up on the screen in the report (even as you use it)
 - Check how the text looks in the methods pages
 - Test all the data + subset of data in the results pages and check that everything is working ok
 - Refresh the page so that any changes you have made into the code are uploaded
 - Sometimes, when the reports are being locally deployed like this, error messages may initially pop up but if you only refresh the page and try again, they no longer appear (some time of initialisation problem, does not occur when they are deployed via cPanel)
 - See Troubleshooting Guidance in the slides at the end of this presentation for advice on common problems
- 4) Once everything is working properly, you are ready to deploy

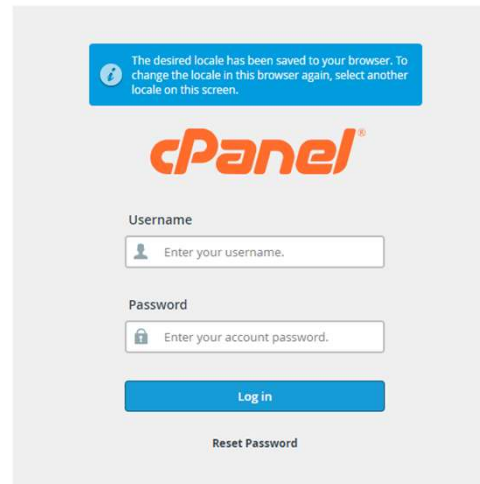
```

C:\WINDOWS\system32\cmd.exe
(dash_app_login_env) C:\Python-Dash-App-and-cPanel-workflow-to-automate-metabolomics-data-analyses-and-visualisation\AppCode\ExampleHomeDirectoryForApp>python app.py
Dash is running on http://127.0.0.1:8050/

* Serving Flask app 'app'
* Debug mode: on
  
```

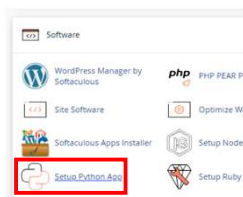
Upload Additional Reports to cPanel : Log into cPanel

- Log into cPanel
 - <https://chostt.is.ed.ac.uk:2083/logout/?locale=en>



Upload Additional Reports to cPanel : Check that the deployed app is running ok

Navigate to the Setup Python App section and select it



Upload Additional Reports to cPanel : Check that the deployed app is running ok

Select the App URI/link to check that it the app is actually running ok before you initialise any changes

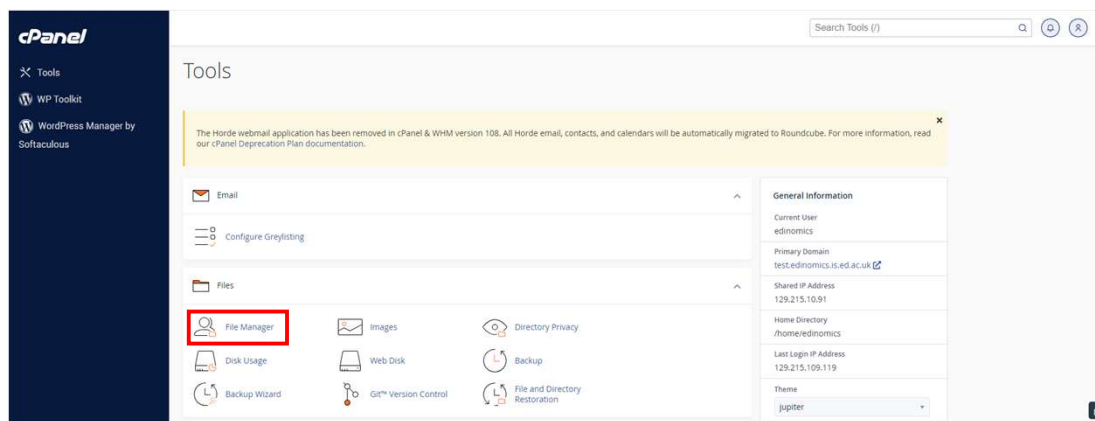
WEB APPLICATIONS CREATE APPLICATION

App URI	App Root Directory	Status	Actions
test.edinomics.is.ed.ac.uk/	/home/edinomics/public_html/20250407_LoginPage_AllData	started (v3.9.23)	   

Whilst you are still getting used to deploying apps, I'd recommend having a backup of the compressed zip files containing all the data currently deployed from File Manager saved on your computer so that, if this next step fails for any reason and the app needs to be reset by IT, you still have the data ready to go

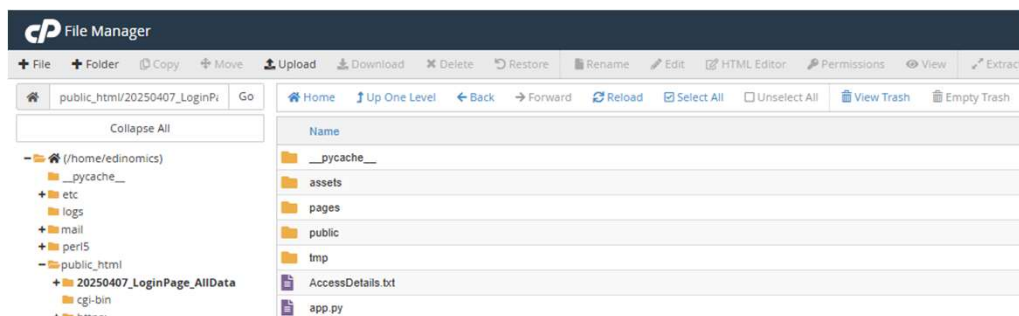
Upload Additional Reports to cPanel : Navigate to cPanel File Manager

- Go to Tools > Files > File Manager



Upload Additional Reports to cPanel : Navigate to Application Root

- In File Manager, find the folder containing the main app.py



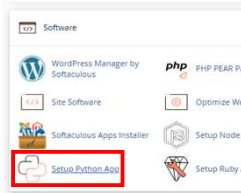
Upload Additional Reports to cPanel : Upload the following updated/new files

EdinOmics Reports

requirements.txt	
Datasets	← Upload new CSV files (metabolite annotations.csv + SampleLabel.csv)
app.py	← Upload the updated version of app.py (only if you have added a new user account)
AccessDetails.txt	← Upload the updated version of AccessDetails.txt (only if you have added a new user account)
assets	
pages	
Login.py	
Logout.py	
UserHomePage.py	← Upload new version of this script if adding a <i>new</i> user account or upload the updated version of this script if only updating an <i>existing</i> user account
[individual .py pages]	← Add additional methods-page.py and results-page.py

Upload Additional Reports to cPanel : Reset Deployed app to incorporate changes

Navigate to the Setup Python App section and select it



Upload Additional Reports to cPanel : Reset Deployed app to incorporate changes

To incorporate the new changes, select the “Restart the application” button

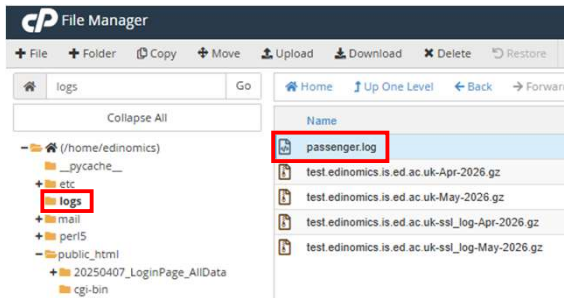
WEB APPLICATIONS CREATE APPLICATION

App URI	App Root Directory	Status	Actions
test.edinomics.is.ed.ac.uk/	/home/edinomics/public_html/20250407_LoginPage_AllData	● started (v3.9.23)	■ 🔄 ✎ 🗑

Once reset, click the App URI/link to check that it is working and your new files/accounts are working properly

Upload Additional Reports to cPanel : See error logs to help troubleshoot problems

If any problems arise, the error log (scroll to the bottom of it) may have information that can help you.



Note: the compressed files in the logs folder can be deleted without any problem

If the passenger.log file is really massive, then you can delete it and restart the app again to generate a fresh error log

See Troubleshooting Guidance at the bottom of this presentation for advice on common problems

Troubleshooting Guidance: Page not displaying (& no error message)

- Usually due to paths not being correctly declared or linked together
 - Check href links
 - Print out page details & check they are correct

Troubleshooting Guidance:

Cannot deploy first instance of python app

- May be due to files from previous apps being present + conflicting
- Try removing these
- If this does not work: back up files + contact IS Helpline to get cPanel reset